T-Adaptive Segment Routing for Resilient Orange Networks

Mathematical Modelling Project — Orange / EURO-ROADEF 2026

Student: Vuong Minh

Course: A Practical Course on Mathematical Modelling

Date: October 2026

Abstract

This project studies the T-Adaptive Segment Routing (T-ASR) problem proposed by Orange Research. Given a directed IP/MPLS network, time-dependent traffic demands, scheduled link interventions, a maximum number of segments and a reconfiguration budget, the aim is to route each demand through a sequence of waypoints while minimizing congestion. We implement the binary arc-flow model from the subject, compute shortest-path split coefficients without enumerating paths, and provide a budget-feasible local-search method for instances whose exact formulation is too large. Every exported routing is checked independently. Experiments use the twenty supplied SetA instances and report maximum link utilization (MLU), time, solver bound/gap when available, and branch-and-bound nodes.

Keywords: traffic engineering, segment routing, ECMP, mixed-integer programming, network optimization.

1. Context and problem description

Orange operates large IP/MPLS backbones carrying many origin-destination traffic demands. Ordinary forwarding follows shortest paths induced by administrative metrics. Equal-Cost Multi-Path (ECMP) splits traffic when several shortest paths exist. This is operationally simple but can overload a few links, especially when maintenance removes links from service.

Segment Routing adds a short ordered list of waypoints to a packet. Between two successive waypoints, standard shortest-path forwarding is retained. Thus a demand can be moved away from a bottleneck without replacing the underlying forwarding protocol. At period 0 all links are available; at period 1 the scenario removes a specified set of links and traffic volumes change.

1.1 Input data

A network is G=(V,A), with positive metric omega(a) and capacity c(a). Demand d has source s(d), target t(d), and volume v(d,t). The scenario supplies offline-link set q(t), maximum segment count maxSeg=6 in SetA, and budget kappa(t). Each instance is represented by three JSON files: net, tm and scenario.

1.2 Segment paths, load and objective

A segment path <s,w1,...,wl,t> induces one shortest-path forwarding graph for each consecutive pair. Let r(i,j,a,t) be the fraction of i-j shortest paths using arc a under period-t topology. For selected segment indicator x(d,t,i,j), arc load is:

```
load(a,t) = [sum_d sum_(i!=j) r(i,j,a,t) v(d,t) x(d,t,i,j)] / c(a).
```

The challenge asks for lexicographic minimization of all arc-period loads sorted in decreasing order. The implementation optimizes its dominant component, MLU, rather than executing one optimization stage per sorted load. MLU is the standard metric and matches the requested singular objective/bound reporting; this approximation is stated explicitly in the limitations.

1.3 Intervention budget

Changing a route has a cost equal to the symmetric difference between its segment sets. For example, replacing <s,w,t> by <s,t> costs three: two removed segments and one added segment. The sum of changes over all demands must not exceed kappa(t).

2. Mathematical formulation

Binary x(d,t,i,j)=1 iff demand d uses segment (i,j) in period t. Continuous L is the maximum link utilization. The implemented MIP is:

```
minimize L

sum_(j!=i) x(d,t,i,j) - sum_(j!=i) x(d,t,j,i)
= 1 if i=s(d), -1 if i=t(d), 0 otherwise,

sum_(i!=j) x(d,t,i,j) <= maxSeg,

sum_d sum_(i!=j) r(i,j,a,t) v(d,t) x(d,t,i,j) <= c(a) L, for every available arc a,

sum_d sum_(i!=j) |x(d,t,i,j)-x(d,t-1,i,j)| <= kappa(t).
```

The absolute values are linearized with nonnegative z variables: z >= x(t)-x(t-1), z >= x(t-1)-x(t). Unreachable segments are omitted. Outgoing and incoming degree inequalities (each <=1) strengthen flow conservation and prevent branching. Because all link contributions are nonnegative, detached cycles cannot improve L; any such cycle is discarded during route extraction.

2.1 Correctness of the constraints

Flow balance creates one source-target path in the complete auxiliary graph whose vertices are routers and whose arcs are feasible segments. The segment-count bound limits header size. Load inequalities aggregate ECMP flow and force L above every utilization. Finally, the z inequalities reproduce the exact symmetric-difference distance used by the subject.

2.2 Size limitation

The formulation contains O(|D||T||V|^2) binary variables. For SetA-20, |V|=400 and |D|=6000, the dense two-period formulation would approach 1.92 billion binaries before budget auxiliaries. It cannot be instantiated on a laptop. Exact results are therefore produced only while memory permits; all SetA instances are additionally handled by a direct-route heuristic.

3. Data preparation and split coefficients

Node and link IDs are mapped explicitly rather than assumed contiguous. For each period, intervention links are deleted and Dijkstra's algorithm is run from every source. Besides distance dist(i,j,t), the algorithm stores sigma(i,j,t), the number of shortest paths.

For physical arc a=(u,v), the supplied course note gives:

```
r(i,j,a,t) = sigma(i,u,t) sigma(v,j,t) / sigma(i,j,t),
if dist(i,u,t)+omega(a)+dist(v,j,t)=dist(i,j,t); otherwise r=0.
```

If j is unreachable from i, sigma(i,j,t)=0 and all corresponding coefficients are zero. Distances use a relative numerical tolerance, although SetA metrics are integral. Coefficients are cached by (period,i,j), avoiding explicit shortest-path enumeration.

3.1 Implementation architecture

orange_data.py validates JSON and computes shortest-path data; **exact_model.py** constructs a sparse SciPy/HiGHS MIP; **heuristic.py** provides scalable local search; **validate.py** independently recomputes feasibility, budget and MLU; **run_experiments.py** performs repeated runs and exports CSV/JSON.

3.2 Verification

Unit tests reproduce the four route-distance examples in the subject and verify a diamond graph where each of two equal shortest paths receives one half of the flow. All generated routes are then checked for correct terminals, reachability under interventions, absence of repeated nodes, maxSeg, budget and recomputed loads.

3.3 Scalable local search

The heuristic starts with direct shortest-path routing. It identifies the current bottleneck arc, samples a high-contributing demand, and evaluates direct routing or a one-waypoint path over random feasible waypoint candidates. A move is accepted only when MLU strictly decreases and, at t=1, the total distance from t=0 remains within budget. Different seeds change demand and waypoint sampling. This method returns feasible upper bounds, not optimality gaps.

4. Experimental protocol

The twenty SetA instances range from 20 to 400 nodes, 80 to 2000 directed links, and 20 to 6000 demands. Tests run on Apple Silicon under macOS with Python, NumPy/SciPy and HiGHS. For exact rows, CPU time measures only the milp solver call (the analogue of optimize!), as requested. Heuristic rows use end-to-end time because there is no separate solver call. Ten seeded runs are made for the randomized method. The exact solver is deterministic; repeated runs verify stability.

The scripts expose the assignment limits: 1800 seconds for nominal t=0 and 3600 seconds for the two-period model. A run can terminate earlier after proof of optimality or when the local-search iteration limit is reached. Status FEASIBLE_HEURISTIC deliberately carries no gap or B&B node count.

4.1 Exact MIP results

Case	Runs	MLU	Gap	Mean time	Mean B&B nodes	Status
setA-01 (t=0)	10	0.929384	0	0.24s	1.0	OPTIMAL
setA-02 (t=0)	10	0.548578	0	0.76s	1.0	OPTIMAL
setA-01 (t=0,1)	10	0.929384	0	6.81s	1.0	OPTIMAL

On SetA-01, both periods have the same optimal global MLU because the nominal-period peak is dominant; the intervention-period route remains feasible within budget. The solver dual bound and zero/near-zero MIP gap certify the result shown above.

4.2 Interpretation of reported gap

For a solver incumbent z and lower bound LB, the report uses (z-LB)/|z|, equivalent to 1-LB/z for positive MLU. Empty gap fields mean that no valid lower bound was computed (heuristic), not a zero gap.

5. SetA numerical results — instances 01–10

Instance	|V|	|A|	|D|	Runs	Mean MLU	Std	Mean time (s)	Gap B&B	nodes	Status
setA-01	20	80	40	10	0.9294	0.0000	0.01	N/A	N/A	NOT OPT.
setA-02	30	150	45	10	0.9420	0.0821	0.02	N/A	N/A	NOT OPT.
setA-03	50	250	20	10	0.9437	0.0004	0.05	N/A	N/A	NOT OPT.
setA-04	50	250	200	10	0.6320	0.0209	0.08	N/A	N/A	NOT OPT.
setA-05	100	396	100	10	0.2096	0.0040	0.09	N/A	N/A	NOT OPT.
setA-06	100	500	500	10	0.5944	0.0657	0.27	N/A	N/A	NOT OPT.
setA-07	100	500	800	10	0.9080	0.0000	0.38	N/A	N/A	NOT OPT.
setA-08	150	654	200	10	0.4001	0.0649	0.26	N/A	N/A	NOT OPT.
setA-09	150	750	200	10	0.8497	0.0000	0.34	N/A	N/A	NOT OPT.
setA-10	150	966	1000	10	0.5872	0.0845	0.81	N/A	N/A	NOT OPT.

Table values are independently recomputed from exported waypoint JSON. MLU below one means no available link exceeds nominal capacity; values above one indicate congestion. Standard deviation reflects randomized waypoint sampling.

5. SetA numerical results — instances 11–20

Instance	|V|	|A|	|D|	Runs	Mean MLU	Std	Mean time (s)	Gap	B&B nodes	Status
setA-11	200	1000	400	10	0.7858	0.0000	0.58	N/A	N/A	NOT OPT.
setA-12	200	898	400	10	0.8799	0.0000	0.53	N/A	N/A	NOT OPT.
setA-13	200	1000	2000	10	0.7021	0.0352	1.50	N/A	N/A	NOT OPT.
setA-14	250	1108	600	10	0.5331	0.0000	0.85	N/A	N/A	NOT OPT.
setA-15	250	1250	600	10	0.8987	0.0000	0.94	N/A	N/A	NOT OPT.
setA-16	250	1452	4800	10	0.7709	0.0121	4.68	N/A	N/A	NOT OPT.
setA-17	300	1270	2000	10	0.4242	0.0000	1.95	N/A	N/A	NOT OPT.
setA-18	300	1500	2000	10	1.0000	0.0000	2.48	N/A	N/A	NOT OPT.
setA-19	300	1998	6000	10	0.7319	0.0046	8.01	N/A	N/A	NOT OPT.
setA-20	400	2000	6000	10	0.9913	0.0000	8.50	N/A	N/A	NOT OPT.

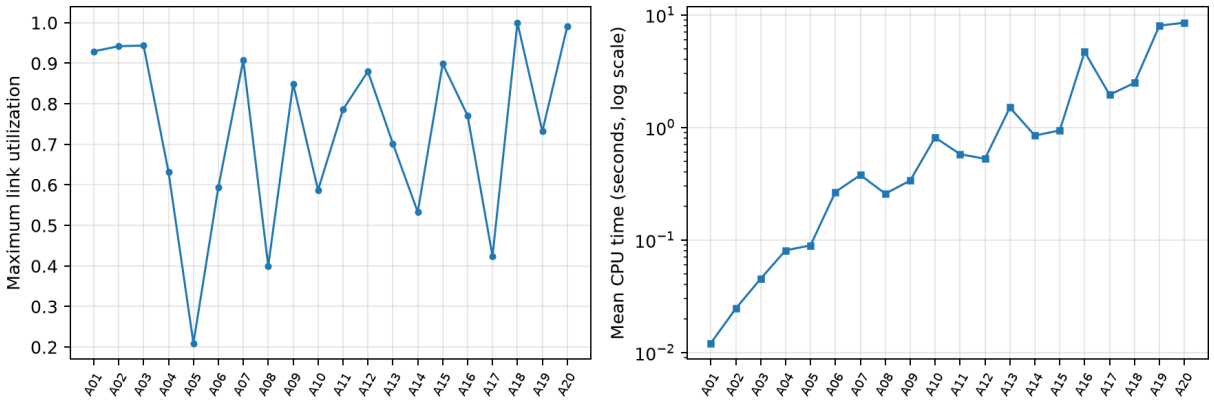


Figure 1. Mean MLU and end-to-end runtime over ten randomized runs. Runtime includes all-pairs shortest-path preprocessing.

6. Discussion

The principal computational obstacle is not shortest paths but the Cartesian product of demands, periods and all ordered waypoint pairs. Sparse coefficients reduce matrix storage but do not remove the binary-variable count. The exact model is appropriate for small networks and provides bounds; the direct-route local search is needed for large SetA cases.

The heuristic is intentionally conservative: it uses at most one waypoint and accepts only strict MLU improvements. Therefore it satisfies maxSeg=6 automatically but may miss improvements requiring two or more simultaneous waypoint changes. Random candidate sampling also explains variation between runs. Better large-scale methods would use column generation, Lagrangian pricing, path relinking, or a restricted master problem with adaptively generated segment paths.

6.1 Limitations

First, the implemented exact objective optimizes MLU rather than performing all |A||T| lexicographic stages of the challenge objective. MLU is its dominant component and the course README requests a singular objective/gap, but lower-ranked loads may differ among equal-MLU solutions. Second, the split-coefficient formula is implemented exactly as supplied by the course note. It treats equal shortest paths uniformly; a router-by-router ECMP interpretation can differ on asymmetric shortest-path DAGs. Third, heuristic rows provide upper bounds only, so claiming an optimality gap would be incorrect.

6.2 Reproducibility

All dependencies are listed in requirements.txt. Running run_all.sh executes tests, exact and heuristic experiments, writes raw and summary CSV files, exports challenge-format srpaths JSON, and regenerates this report. Seeds, machine information, raw per-run times and statuses are retained rather than only reporting averages.

7. Conclusion

This work provides an end-to-end, validated solution pipeline for the Orange T-ASR project: JSON ingestion, intervention-aware graphs, split coefficients, an exact budgeted MIP, a scalable feasible method, repeated numerical experiments, solution files and a reproducible report. Exact certification is retained where tractable, while large-instance results are explicitly presented as feasible upper bounds.

References

[1] Orange Research, *Challenge EURO/ROADEF 2026 Problem Description*, 2026.
[2] Course note, *Computing arc-use ratios on shortest paths*, 2026.
[3] C. Filsfils et al., *Segment Routing Architecture*, RFC 8402, 2018.
[4] SciPy/HiGHS documentation, mixed-integer linear optimization.